

Chameleon Edge, explained

A security guard for web apps that runs on one small HP computer. It spots attackers, keeps them busy with convincing fake data, and meanwhile teaches itself a fix so the same trick never works again.

The problem

Most security tools only **block** an attack. The attacker learns they were caught, changes the attack slightly and tries again, and the defenders learn almost nothing. Fixing each new hole by hand takes people and time.

Our idea

Don't just block. **Redirect** the attacker to a fake copy of the app, let them think they succeeded, record everything, and let AI on the device **write, test and install a patch** for that attack while they're busy. The next attempt is blocked.

In five steps

- 1 Check.** Every request is scored by Jev (a cloud decision model) and by our own model on the Nano.
- 2 Trick.** Attackers are silently sent to a decoy app that answers with AI-written fake data.
- 3 Learn.** The AI writes a fix, runs four tests, and a second AI tries to break it.
- 4 Block.** The approved fix is live at once, and written into the app's own code.
- 5 Measure.** The same AI work runs on AWS too, so edge and cloud cost can be compared.

Measured results

0% → 78%

an attack type the system had never seen, caught after it learned (280 attacks, 0 false alarms on 6,434 normal requests)

13 / 13

live requests routed correctly; Jev caught 5 attacks our Nano model alone would have missed

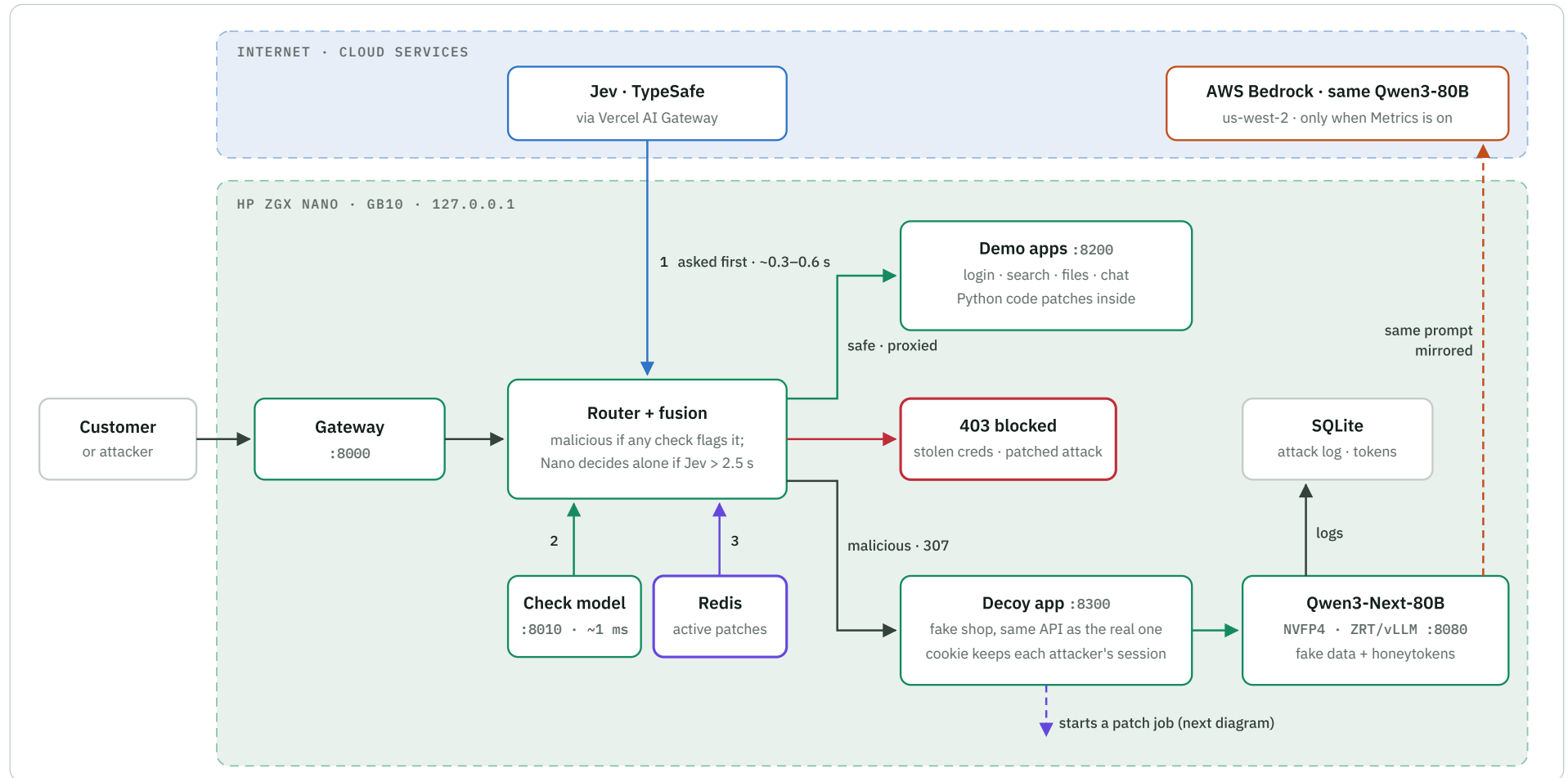
~59× cheaper

per request on the Nano than on AWS Bedrock for the same model and prompt (electricity only)

~40 s – 3 min

from an attack arriving to a tested patch blocking it

What happens to each request



Three checks, in order

Jev (TypeSafe's model, reached through Vercel AI Gateway) answers first: how likely this is an attack, and what kind. Then our **check model** on the Nano, then the **patches** learned so far. If any says "attack", it's treated as one. If Jev is slow or down, the Nano decides alone.

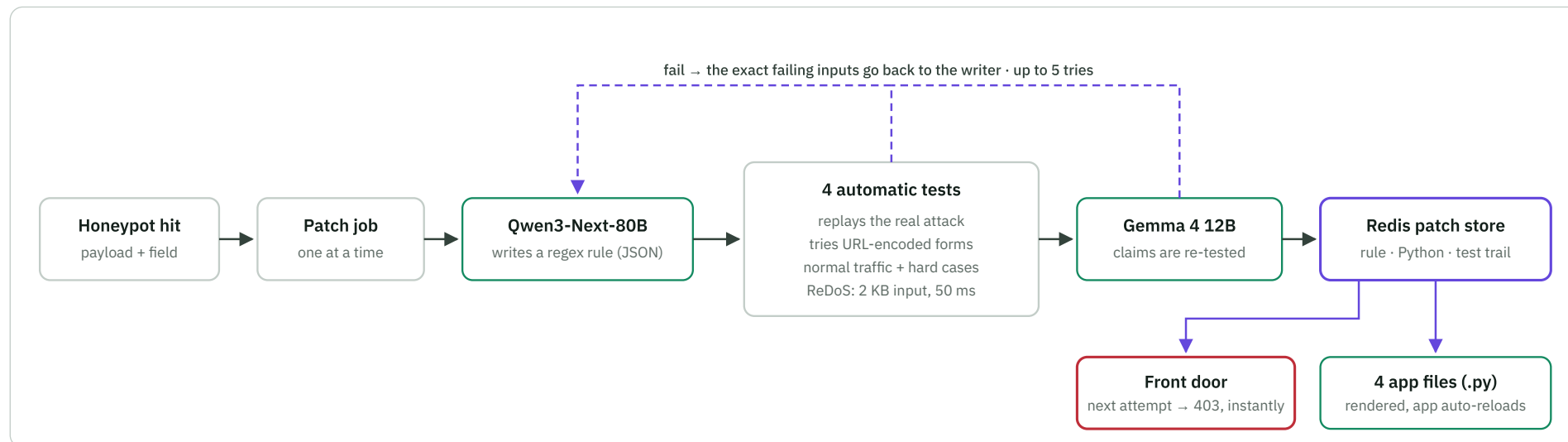
Three outcomes

Safe: passed straight to the real app. **Known attack or stolen password:** refused with a 403. **New attack:** a 307 redirect sends the attacker's browser to the **decoy app**, which looks and answers like the real one.

The decoy's fake data

Qwen3-Next-80B on the Nano writes each answer live, so it's never the same twice. Each attacker also receives **unique fake passwords** (honeytokens). If one is ever used, we know exactly which attacker took it.

How an attack becomes a fix



Write

Qwen3-Next-80B reads the captured attack and writes a **rule**: a text pattern (a regular expression) that matches this kind of attack.

Test, then challenge

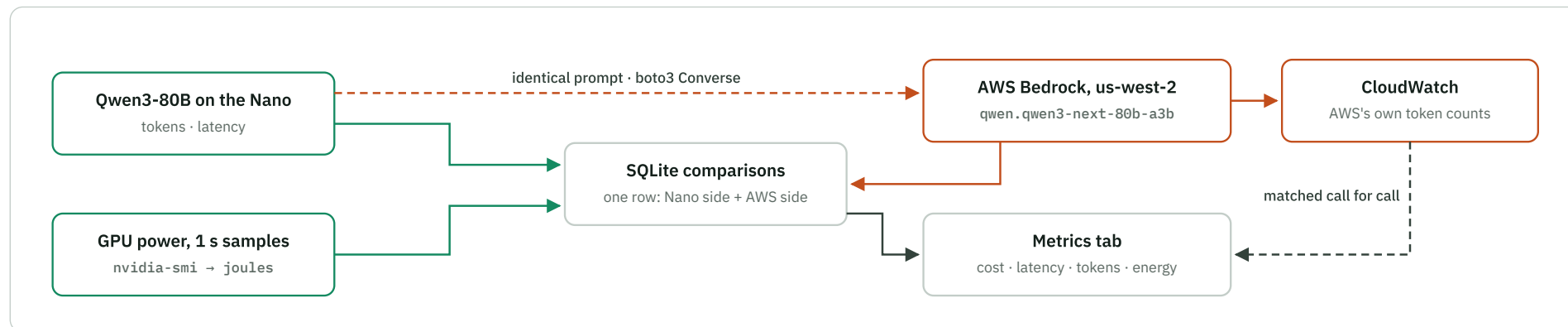
Four automatic tests: does it block the real attack, its disguised (URL-encoded) forms, and **none** of a set of normal customer inputs, and is it fast even on huge input? Then Gemma 4 12B, a different AI, looks for ways around it; its claims are re-tested. Failures go back to the writer, up to 5 tries.

Install

The approved rule is saved in **Redis** with its full test history. The front door uses it on the very next request, and it's also written into the app's Python code between marked lines, so the app refuses the attack even if someone bypasses the front door.

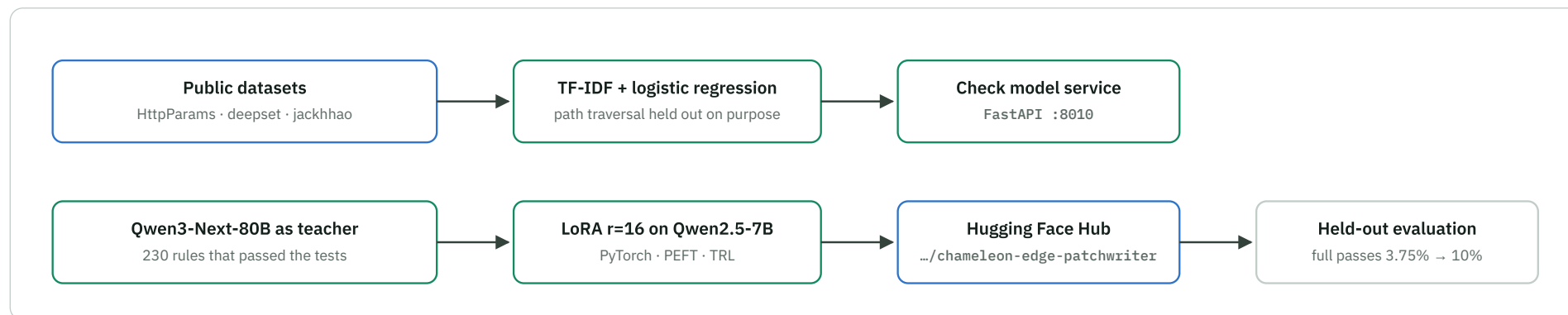
Safety by design: the AI only chooses the pattern, never the code. The code is filled in from a fixed template, and each file must compile before it replaces the old one. A pattern that would take too long is cut off after 50 ms and treated as a match.

Measuring the Nano against AWS



Every time the Nano's AI does work, the identical prompt is also sent to **AWS Bedrock** running the same model. Both results are stored side by side: tokens, response time, cost and (on the Nano) energy from the GPU's power meter. AWS's own CloudWatch counts matched ours call for call. In our run the Nano was about **59× cheaper per request** on electricity, AWS was about **2× faster** per request, and buying a Nano pays off above roughly **5,800 attacks a day**, up to about 80,000 a day per Nano.

What was trained



The technology

LAYER	TECHNOLOGY	WHAT IT DOES HERE	RUNS ON
Hardware	HP ZGX Nano · NVIDIA GB10 Grace Blackwell, 128 GB unified memory, aarch64	Runs every model and service below	Nano
Model serving	HP ZRT wrapping vLLM 0.26, one proxy on :8080	Serves both LLMs; GPU memory split 45% / 25%, 8K context	Nano
Decoy + patch writer	Qwen3-Next-80B-A3B-Instruct, NVFP4	Writes the decoy's fake answers and every patch rule	Nano
Patch verifier	Gemma 4 12B (google/gemma-4-12B-it)	A different model family hunts for bypasses in each rule	Nano
Check model	scikit-learn 1.9 · TF-IDF + logistic regression	Scores every request in about 1 ms	Nano
Decision API	TypeSafe Jev via Vercel AI Gateway (typesafe-ai/jev)	First check on every request, with the attack type	Cloud
Services	Python 3.12 · FastAPI 0.141 · Uvicorn · Pydantic · httpx	Gateway :8000, apps :8200, decoy :8300, dashboard :8100	Nano
Rule engine	regex module, 50 ms match timeout, fails closed	Enforces patches; a slow pattern can't take the site down	Nano
Patch store	Redis 8.10, built from source, append-only persistence	Patches, their Python code and full test trails	Nano
Logs	SQLite	Requests, sessions, honeytokens, patch jobs, metric pairs	Nano
Dashboard	One HTML file · vanilla JS · WebSocket · inline SVG chart	HOME (attack console, live patches) and METRICS tabs	Nano
Cloud comparison	AWS Bedrock qwen.qwen3-next-80b-a3b · boto3 Converse · CloudWatch · least-privilege IAM	Same prompt in the cloud for cost and latency; AWS's own counts cross-check ours	AWS
Energy	nvidia-smi power draw, sampled every second	Joules and electricity cost per request	Nano
Fine-tuning	PyTorch 2.11 (CUDA 13) · Transformers · PEFT LoRA · TRL · Accelerate	Distilled a 7B patch writer from the 80B	Nano
Model hosting	Hugging Face Hub (Yukta3030/chameleon-edge-patchwriter)	Public home of the LoRA adapter and model card	Cloud
Training data	HttpParamsDataset · deepset/prompt-injections · jackhhao/jailbreak-classification	Labelled attacks and normal traffic for the check model	Cloud → Nano
AI SDK example	Vercel AI SDK ai 7 on Node 22 · openai/gpt-5-nano	Gateway smoke test in index.ts	Cloud
Testing	pytest (108 tests) · fakeredis · Selenium + headless Firefox	Unit tests and scripted dashboard walkthroughs	Nano
Access & ops	Tailscale · SSH tunnels · tmux · GitHub	Remote access, long-running services, source control	Dev

Terms used in this document

HP ZGX Nano: a desktop-sized AI computer with an NVIDIA GB10 chip and 128 GB of memory. Everything here runs on one.

Jev: TypeSafe AI's decision model. Given a request, it returns the probability that it's an attack and the attack type.

Honeypot / decoy: a fake copy of the app for attackers, so they waste time on worthless data while we watch.

Redis: a fast database, used here to store every patch and its test history.

Edge: running AI on your own hardware, on-site, instead of renting it in the cloud.

Check model: a small, fast classifier we trained on the Nano; it scores a request in about a millisecond.

Honeytoken: a fake password or key planted in the decoy's answers. Anyone who uses it gives themselves away.

307 / 403: web response codes. 307 means "go to this other address"; 403 means "forbidden".

LLM: a large language model that reads and writes text. We run two: Qwen3-Next-80B and Gemma 4 12B.

Gateway: the front door every request passes through; it decides where the request goes.

Patch / rule: a pattern that recognises one kind of attack so it can be refused.

AWS Bedrock: Amazon's cloud AI service. We ran the same Qwen model there to compare cost and speed.

Known limits, measured

- A few attacks still get through both checks, e.g. shoes && id and `{{7*7}}`.
- Jev once sent a real customer message ("Please ignore my last message...") to the decoy.
- Some approved patches have a known bypass that Gemma found; it's recorded, not hidden.
- Decoy answers that invent a whole file can take up to 25 s.
- All data is public benchmark or synthetic data, from one day on one Nano.

Links

- Code: github.com/prags8585/EdgeA1
- Fine-tuned model: huggingface.co/Yukta3030/chameleon-edge-patchwriter
- Everything above runs live on the Nano: dashboard with HOME (live attacks and patches) and METRICS (Nano vs AWS) tabs.